

6.1 Theory: Multi-Layer Perceptron with Backpropagation

Part (a): Vanishing Gradient Problem

What is it?

When we train a deep network using backpropagation, gradients of the loss travel backward through layers to update weights. In deep networks, these gradients tend to get multiplied by small numbers repeatedly and after enough layers, they become so tiny that the early layers essentially stop learning. This is the **vanishing gradient problem**.

Why Does It Happen?

Take a network with L layers using sigmoid activations. The gradient flowing back to layer l involves a chain of partial derivatives:

$$\frac{\partial J}{\partial w^{(l)}} \propto \prod_{k=l}^L w^{(k)} \cdot \sigma'(z^{(k)})$$

The sigmoid function and its derivative are:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \sigma'(z) = \sigma(z)(1 - \sigma(z))$$

The maximum value of $\sigma'(z)$ is attained at $z = 0$:

$$\sigma'(z) \leq 0.5 \times 0.5 = 0.25$$

So every layer multiplies the gradient by **at most 0.25**. Over L layers this gives:

$$\left| \frac{\partial J}{\partial w^{(l)}} \right| \leq \left(\frac{1}{4} \right)^{L-l} \cdot \prod_k |w^{(k)}|$$

If weights are also small (common with standard initialisation), the product shrinks even faster. For a 10-layer network the multiplying factor is roughly $0.25^{10} \approx 10^{-6}$ — the gradient has essentially vanished before it reaches the first layer.

The deeper the network, the worse this gets. Early layers receive almost no learning signal while later layers update normally, creating an uneven, broken training process.

Part (b): Random Graph Model with Dropout ($p = 0.5$)

The Setup

Instead of a standard fully connected network, imagine each hidden unit is randomly switched off with probability $p = 0.5$ during each forward pass. The question is whether this changes how the chain rule works during backpropagation.

Short Answer: The Chain Rule Itself Does Not Change

The chain rule is a mathematical rule for differentiating compositions of functions — it always applies the same way. What changes is the **graph it is applied to**.

When units are dropped, each hidden unit j in layer k receives a random binary mask:

$$m_j^{(k)} \sim \text{Bernoulli}(0.5)$$

The masked output of unit j becomes:

$$\tilde{a}_j^{(k)} = m_j^{(k)} \cdot \sigma(z_j^{(k)})$$

During backpropagation, the chain rule is applied only to the **active subgraph** — the units that were not dropped. For any unit where $m_j^{(k)} = 0$, the gradient is zero; it contributes nothing and its weights are not updated in that step.

The gradient through surviving units is:

$$\frac{\partial J}{\partial w^{(l)}} = \sum_{\text{active paths}} \prod_{k=l}^L m^{(k)} \cdot w^{(k)} \cdot \sigma'(z^{(k)})$$

How Does This Affect the Vanishing Gradient?

Random dropping can actually **reduce** the vanishing gradient problem to some extent. With $p = 0.5$, roughly half the units are dropped per layer, so any given gradient path through the network has fewer active multiplications. Fewer multiplications of small numbers means the gradient does not shrink as aggressively as it would in the fully connected case.

Over many training steps, the expected gradient averages across all randomly sampled sub-graphs:

$$\mathbb{E}_m \left[\frac{\partial J}{\partial w^{(l)}} \right] = \sum_{\text{all paths}} (0.5)^{L-l} \cdot \prod_{k \in \text{path}} w^{(k)} \cdot \sigma'(z^{(k)})$$

The factor $(0.5)^{L-l}$ reflects the probability that a full path of length $L-l$ survives the random dropping.